required if running a web server that uses threads rather than processes for workers). The installation package is simply a `.zip` file containing the dozens of binaries (`.exe` and `.dll` files). Extract the contents to the directory of your choice (`C:\PHP` will be fine) and make sure that directory is part of your path, and you can simply invoke PHP from the command line like the examples above. The default default command-line environment for Windows (`cmd.exe`) is a little clunky, I highly recommend installing https://gitforwindows.org/ which bundles a nicer terminal GUI with a wide variety of GNU binaries that emulate most common *nix shell commands.

There is also the option of WSL (Windows Subsystem for Linux) which utilizes a virtual machine hypervisor to run a choice of Linux distributions, see https://learn.microsoft.com/en-us/windows/wsl/install to get started. There is of course the slightly more traditional option of running a VM via VirtualBox (or similar) https://www.virtualbox.org/wiki/Downloads but WSL is likely to be less hassle in the long run. Finally, there is Docker, which is the preferred method of running the code examples throughout this book. Read on for everything you'll need to get Docker up and running, giving you a powerful platform-agnostic execution environment.

PHP Extensions

PHP is modular: there is a core of functionality which is always available, and there are extensions which introduce new things into the language. For example, if your application will never interact with an LDAP server, there is no need for the LDAP functions (like `ldap_connect()`) or classes (in the `LDAP` namespace) to be installed. Every function, class, or constant defined in PHP takes system resources. In an environment where every millisecond counts (you might have a time budget measured in hundreds of milliseconds for a request to be fully processed) it makes sense to pare the interpreter down to the bare minimum.

To confirm that an extension (aka module) is enabled (or not) in your environment, simply run `php -m` to output the list:

```
$ php -m
[PHP Modules]
calendar
Core
ctype
date
...
```